GOLD: Indefinite Loops (Wed Sep 9)

Midterm Announcement

Midterm dates have been scheduled:

- Midterm 1: Wednesday, September 30
- Midterm 2: Wednesday, October 21
- Midterm 3: Wednesday, November 11
- Midterm 4: Wednesday, December 2

There will be two time slots: 6:30 - 7:30 pm and 8:00 - 9:00 pm. You will be assigned to one of those two slots to minimize conflicts with other tests and classes.

There will be no lecture on a midterm night.

Each midterm will have an optional retake the following week. We will announce dates for those once we've scheduled the computer labs.

Introducing While

What's Missing? Lots of Data

- You know how to make a decision and execute one statement or another
- Problems up to now involved working on a fixed set of input or small number of values
- Next power up: perform repetitive actions
- Two ways to think about repetition:
    - Perform the same operation on different data
        - Produce receipts for 300 orders
    - Accumulate information over a set of data
        - Add up 300 variables

Java's syntax for a repetition uses the keyword `while`:

```
if (boolean-expression) {
    then-statement1;
    ...
}
next-statement

while (boolean-expression) {
    loop-statement1;
    ...
}
next-statement;
```

One statement, no braces version:

```
if (boolean-expression)
    then-statement;
next-statement;

while (boolean-expression)
    loop-statement;
next-statement;
```

Good style for CS 18000: **always use the braces** on a loop body, even for a single statement. Adding a second line to the loop body later is the most common way the braceless form goes wrong.

Repetition

Repetition Concept

```
while (boolean-expression)
    loop-statement;

next-statement;
```

```
while (n > 0) {
    IO.println("Countdown: " + n);
    n = n - 1;
} IO.println("Blast Off!");
```

- Repetition broken into two parts
    - Body of code that gets repeatedly executed (`loop-statement`)
    - Condition (boolean) to determine when to stop (`boolean-expression`)
- How to construct the body so that it does something different/useful each time it is run?
- The state of the computation must change with each iteration (otherwise, nothing new is done except waste time)

Two Forms of Iteration (looping)

- Indefinite: loop as long as expression that means "not Done" is true
    - no advance knowledge of how many iterations will be executed
- Definite: loop a "given" number of times
    - used when the number of iterations executed is controlled by a counter or size or limit

The while Loop

```
while (boolean-expression) {
    statements;  // while body
}                // end of while body
next_statement;           // first statement after while body
```

- Evaluate `boolean-expression`
- If (expression == TRUE) execute `statements;` else skip beyond while body to execute `next_statement;`
- Upon reaching end of while body, skip back to keyword while and resume execution

```
void main() {
    int n = 3;
    while (n > 0) {
        IO.println("Countdown: " + n);
        n = n - 1;
    }
    IO.println("Liftoff!");
}
```

Tracing Loops

Trace it on paper

You will understand loops only when you can run one **by hand**.

```
void main() {
    int n = 3;
    while (n > 0) {
        IO.println(n);
        n = n - 1;
    }
    IO.println("Liftoff!");
}
```

Make a table tracking all variables, the boolean condition, and any output or actions.

Update a row each time through (pass 1, pass 2, etc):

pass	`n`	`n > 0`?	prints
1			

▶ Expand

Pause and predict

Q. Same countdown loop, but now the test is `>=`:

```
int n = 3;
while (n >= 0) {
    IO.println(n);
    n--;
}
```

How many lines print, and what is `n` after the loop?

- **A.** 3 lines, and `n` ends at `0`

- **B.** 4 lines, and `n` ends at `0`
- **C.** 4 lines, and `n` ends at `-1`
- **D.** 3 lines, and `n` ends at `-1`

Watch This Fail

Watch this fail

Here's a loop meant to count down from 3.

```
void main() {
    int n = 3;
    while (n > 0) {
        IO.println(n);
        // (forgot to change n)
    }
}
```

Pause and predict

Q. What happens when this runs?

- **A.** Prints `3 2 1` then `Liftoff!`
- **B.** Prints `3` forever. `Liftoff!` never comes
- **C.** Prints nothing at all
- **D.** Compile error

Remember, a loop must have:

1. **Initialize** the starting variable / state
2. **Test** for when to stop
3. **Update** the state: move towards stopping or get more input

▶ Expand

Example: Summer

Problem: Read a sequence of integers from the standard input and compute their sum and the count of the number of values read.

Two issues:

- How do we know when we're done?
- How do we accumulate the sum?

A **sentinel** is a special data or input value that tells us when to stop.

We don't know how many integers we will get in advance, so let's designate the word `done` as this sentinel.

To read integers from the standard input, we can use the `IO.readln()` method of the IO class to convert an input line to an integer **after** checking for `done`.

Code:

```
void main() {
    int sum = 0;
    int count = 0;

    String response = IO.readln("Enter a number (or \"done\"): ");
    while (! response.equals("done")) {
        int num = Integer.parseInt(response);
        // running total: sum
        sum = sum + num;
        // running count
        count = count + 1;
        response = IO.readln("Enter a number (or \"done\"): ");
    }
    IO.println(String.format("The total is %d from %d numbers entered.",sum,count));
}
```

Above, we accumulate the sum in a variable, and also keep track of the count of numbers entered.

Example: Palindrome

Palindrome

Problem: Implement a method in class `Palindrome`

`boolean isPalindrome(String s)`

to test if s is a palindrome, a string that reads the same backwards as forwards.

Today's Approach: Use a while loop

Strategy:

Edge cases: If string is NULL or length <= 1, return TRUE

While length > 1

- Compare first and last characters
- Differ? End iteration, return FALSE
- Strip off first and last characters
- Skip back to check string length
- return TRUE

Some test inputs:

- “level” (true)
- “henway” (false)
- “henweh” (false)
- “x”, “aba”, “abba” (all true)
- “” (empty string (true))
- null (null value (true))

```
boolean isPalindrome(String s) {
    if (s == null || s.length() < 2)
        return true;

    while (s.length() >= 2) { // "racecar"
        if (s.charAt(0) != s.charAt(s.length() - 1))
            return false;
        s = s.substring(1,s.length() - 1);
    }
    return true;
}

void main() {
    IO.println("Is level a palindrome? " + isPalindrome("level"));
    IO.println("Is henway a palindrome? " + isPalindrome("henway"));
    IO.println("Is e a palindrome? " + isPalindrome("e"));
    IO.println("Is (null) a palindrome? " + isPalindrome(null));
}
```

Break and Continue

Four letter words: `break` and `continue`

Problem: count how many of the words the user enters have exactly four letters (out of all words entered), until the user types `done`.

Program 1: The sentinel in the condition, as we wrote earlier:

```
void main() {
    int fourLetter = 0;
    int count = 0;
    String response = IO.readln("Enter a word (or \"done\" to quit): ");
    while (! response.equals("done")) {
        count = count + 1;
        if (response.length() == 4) {
            fourLetter = fourLetter + 1;
        }
        response = IO.readln("Enter a word (or \"done\" to quit): ");
    }

    IO.println(String.format("Four letter word count: %d out of %d words",
        fourLetter, count));
}
```

Two keywords let us reshape the flow of a loop.

`break`: get me out of here

`break` leaves the loop immediately and continues with the statement after it.

A common use: do the sentinel check **inside** the loop, so the prompt is written only once:

```
void main() {
    int fourLetter = 0;
    int count = 0;
    String response;
    while (true) {
        response = IO.readln("Enter a word (or \"done\" to quit): ");
        if (response.equals("done")) {
            break;                // get me out of here
        }
        count = count + 1;
        if (response.length() == 4) {
            fourLetter = fourLetter + 1;
        }
    }

    IO.println(String.format("Four letter word count: %d out of %d words",
        fourLetter, count));
}
```

`continue`: skip the rest of this pass

`continue` jumps straight back to the loop test, skipping whatever is left of the body.

"Not a four-letter word? Skip it and get the next one":

```
void main() {
    int fourLetter = 0;
    int count = 0;
    String response;
    while (true) {
        response = IO.readln("Enter a word (or \"done\" to quit): ");
        if (response.equals("done")) {
            break;
        }
        count = count + 1;
        if (response.length() != 4) {
            continue;             // not four letters: skip the rest of this pass
        }
        fourLetter = fourLetter + 1;
    }

    IO.println(String.format("Four letter word count: %d out of %d words",
        fourLetter, count));
}
```

- Same program, different shapes.
  - Pick the one that reads most clearly
  - Here that's probably the second version with just `break`

- A `while (true)` loop **must** contain a reachable `break`, or it never ends.

- `continue` is easy to misread: it does not skip the loop, it skips the rest of this pass.

Pause and predict

Q. What is the meaning (in English) of the following code?

```
int count = 0;
while (true) {
    String word = IO.readln("Enter a word: ");
    if (word.length() >= 5)
        break;
    else
        count=count+1;
}
```

- **A.** `count` contains the total number of words of length 4 or less
- **B.** `count` contains the total number of words of length 5 or more
- **C.** `count` contains the total number of words before the first word of length 5 or more
- **D.** `count` contains the total number of words before the first word of length 4 or less

Compound Assignment

Pro Tip: Compound Assignment

- Common assignment statements are extremely common:
  - `x = x + y;`
  - `a = a - b;`
  - `s = s + "\n"; // s is a string`

- Java provides shortcut to save keystrokes:
  - `x += y;`
  - `a -= b;`
  - `s += "\n";`

- Available for all (or most) binary operators

Increment/Decrement Operators

Pro Tip 2: Increment/Decrement Operators

- A refinement for an even more common case:
  - `x = x + 1;`
  - `a = a – 1;`

- Java provides even more keystroke savings:
  - `x++;`
  - `a--;`

- Also:
  - `++x;`
  - `--a;`

Post- and Pre- Increment/Decrement

- `x++` increments x by one, but the expression value is the original x
  - `int x = 0;`
  - `IO.println(x++);`
  - `IO.println(x);`
- Prints 0, then 1

- `++x` increments x by one, and its value is the new x
  - `int x = 0;`
  - `IO.println(++x);`
  - `IO.println(x);`
- Prints 1, then 1

- `x++` and `x--` are very common idioms

- Life becomes messy if an expression contains multiple pre- and post- increment and decrement operators

Pause and predict

Q. Suppose `counter` is currently 4. What is the result of `lastCounter = counter++;`?

- **A.** lastCounter is 4, counter is 4
- **B.** lastCounter is 4, counter is 5
- **C.** lastCounter is 5, counter is 4
- **D.** lastCounter is 5, counter is 5

From While to For

Problem: Print “hello” 10 times using a while loop

We illustrate using a while loop to implement a definite iteration

```
void main() {
    int i = 0;
    while (i < 10) {
        IO.println(String.format("Hello (#%d)", i));
        i++; // increment i by 1, but return old i
    }
}
```

There is another way we could do this:

```
void main() {
    /* Equivalent to:
    int i = 0;
    while (i < 10) {
        System.out.printf("Hello (#%d)\n", i);
        i++;
    }
    */
    for (int i = 0; i < 10; i++) {
        IO.println(String.format("Hello (#%d)", i));
    }
}
```

Take Home: Collatz

Take any positive whole number.

- If it is even, halve it;
- if it is odd, triple it and add one.
- Repeat.

Every number anyone has ever tried eventually reaches **1.**

Nobody has proved it always does, but for this exercise, it does.

```
6 → 3 → 10 → 5 → 16 → 8 → 4 → 2 → 1        (8 steps)
```

Implement

`int stepsToOne(int n)`

that returns **how many steps** it takes `n` to reach 1.

This is an *indefinite* loop in its purest form: you don't know in advance how long that takes.

Press **Test** to run the tests. The boundary to think about first: `stepsToOne(1)` must return **0**.